

Protocol – MoCA Evaluation and Improvement applied for

Case: Ger-CompA

Empirically identified MoCA influences for Ger-CompA:

Low PDI : Scrum Master $\rightarrow \{(1, 1)\}$

Low PDI : Scrum Master $\rightarrow \{(0.75, 1)\}$

Aggregate function for calculating the identified influence:

Low PDI : Scrum Master $\rightarrow \{(1, 1)\}$

```
# Defining the tuples
tuple1 = (1, 1) #Influence from Ger-CompA Case
tuple2 = (0.75, 1) #Influence from Ger-CompA Case

# Calculating the sum of ai*bi
sum_ai_bi = tuple1[0] * tuple1[1] + tuple2[0] * tuple2[1]

# Calculating the sum of bi
sum_bi = tuple1[1] + tuple2[1]

# Calculating the required formula: Sum(ai*bi) / Sum(bi)
result = sum_ai_bi / sum_bi

# Result of the formula: 0.875
```

```
# Calculating Sum bi
sum_bi_new = tuple1[1] + tuple2[1]

# Calculating Sum(ai - 1) * bi
sum_ai_minus_1_bi = (tuple1[0] - 1) * tuple1[1] + (tuple2[0] - 1) *
tuple2[1]

# Calculating Sum ai
sum_ai_new = tuple1[0] + tuple2[0]

# Calculating the new formula
result_new = sum_bi_new - (sum_ai_minus_1_bi / sum_ai_new)
result_new

Result = 2.142857142857143
Rounded to: 2
```

Protocol – MoCA Evaluation and Improvement applied for

Case: Ger-CompB

Empirically identified MoCA influences for Ger-CompB:

Low PDI : Agile Estimation $\rightarrow$ {Management defines estimates : (0.75, 1)}

Low PDI : Empowered and self-organized team $\rightarrow$ {(1, 1)}

Low PDI : Scrum Master $\rightarrow$ {(0.75, 1)}

Low PDI : Product Owner $\rightarrow$ {(1, 1)}

Low PDI : Product Owner $\rightarrow$ {(0.75, 1)}

High IDV : Retrospective Learning Loop $\rightarrow$ {(0.75, 1)}

High IDV : Using and maintaining a backlog $\rightarrow$ Dynamic project environment : {(0.75, 1)}

Input influences for the aggregate function from MoCA v1.3

Aggregate function for calculating the identified influence:

Low PDI : Empowered and self-organized team $\rightarrow$ {(1, 1)}

Defining the tuples

```
tuple1 = (1, 1) #Influence from MoCA V1.3
```

```
tuple2 = (1, 1) #Influence from Ger-CompB Case
```

Python code for calculating the ai value (=m):

```
print(f"Defined tuples: {tuple1}, {tuple2}")
```

```
# Calculating the sum of ai*bi
```

```

sum_ai_bi = tuple1[0] * tuple1[1] + tuple2[0] * tuple2[1]
print(f"Sum of ai*bi: {sum_ai_bi}")

# Calculating the sum of bi
sum_bi = tuple1[1] + tuple2[1]
print(f"Sum of bi: {sum_bi}")

# Calculating the required formula: Sum(ai*bi) / Sum(bi)
result = sum_ai_bi / sum_bi
print(f"Result of Sum(ai*bi) / Sum(bi): {result}")

```

Output:

```

Defined tuples: (1, 1), (1, 1)
Sum of ai*bi: 2
Sum of bi: 2
Result of Sum(ai*bi) / Sum(bi): 1.0

```

Python code for calculating bi:

```

# Defining the tuples
tuple1 = (1, 1)
tuple2 = (1, 1)

# Recalculating with the new formula
# Sum bi - ((Sum(ai - 1) * bi) / Sum ai)

# Calculating Sum bi
sum_bi_new = tuple1[1] + tuple2[1] # 1 + 1
print(f"Sum of bi: {sum_bi_new}") # Sum of bi: 2

# Calculating Sum(ai - 1) * bi
sum_ai_minus_1_bi = (tuple1[0] - 1) * tuple1[1] + (tuple2[0] - 1) *
tuple2[1] # (1-1)*1 + (1-1)*1

```

```
print(f"Sum of (ai - 1) * bi: {sum_ai_minus_1_bi}") # Sum of (ai - 1) * bi: 0
```

```
# Calculating Sum ai
```

```
sum_ai_new = tuple1[0] + tuple2[0] # 1 + 1
```

```
print(f"Sum of ai: {sum_ai_new}") # Sum of ai: 2
```

```
Sum(ai*bi) / Sum(bi)
```

```
result_new = sum_bi_new - (sum_ai_minus_1_bi / sum_ai_new)
```

```
print(f"Result of the formula: {result_new}")
```

```
# Result of the formula: 2.0
```

Aggregate function for calculating the identified influence:

Low PDI : Scrum Master $\rightarrow \{(0.75, 1)\}$

```
# Defining the tuples
```

```
tuple1 = (0.875, 2) #Influence from MoCA V1.3
```

```
tuple2 = (0.75, 1) #Influence from Ger-CompB Case
```

```
# Calculating the sum of ai*bi for the new tuples
```

```
sum_ai_bi_new = tuple1_new[0] * tuple1_new[1] + tuple2_new[0] * tuple2_new[1]
```

```
# Calculating the sum of bi for the new tuples
```

```
sum_bi_new = tuple1_new[1] + tuple2_new[1]
```

```
# Calculating the required formula: Sum(ai*bi) / Sum(bi) for the new tuples
```

```
result_new_tuples = sum_ai_bi_new / sum_bi_new
```

```
Result = 0.83333333333333333334
```

```
Rounded to = 0.83
```

```

# Redefining the tuples and constant value for the calculation
tuple1_new = (0.875, 2)
tuple2_new = (0.75, 1)
constant_value_new = 0.833

# Calculating Sum bi
sum_bi_new_two_tuples = tuple1_new[1] + tuple2_new[1]

# Calculating Sum(ai - constant_value_new) * bi
sum_ai_minus_constant_new_bi = ((tuple1_new[0] - constant_value_new)
* tuple1_new[1] +
                                (tuple2_new[0] - constant_value_new)
* tuple2_new[1])

# Calculating Sum ai
sum_ai_new_two_tuples = tuple1_new[0] + tuple2_new[0]

# Calculating the new formula
result_new_formula_two_tuples = sum_bi_new_two_tuples -
(sum_ai_minus_constant_new_bi / sum_ai_new_two_tuples)
Result = 2.9993846153846153
Rounded to: 3

```

Aggregate function for calculating the identified influence:

Low PDI : Product Owner $\rightarrow \{(1, 1)\}$

Low PDI : Product Owner $\rightarrow \{(0.75, 1)\}$

Defining the tuples

```
tuple1 = (1, 1) #Influence from Ger-CompB Case
```

```
tuple2 = (0.75, 1) #Influence from Ger-CompB Case
```

```
# Calculating the sum of ai*bi
```

```
sum_ai_bi = tuple1[0] * tuple1[1] + tuple2[0] * tuple2[1]
```

```
# Calculating the sum of bi
```

```
sum_bi = tuple1[1] + tuple2[1]
```

```
# Calculating the required formula: Sum(ai*bi) / Sum(bi)
```

```
result = sum_ai_bi / sum_bi
```

```
# Result of the formula: 0.875
```

```
# Calculating Sum bi
```

```
sum_bi_new = tuple1[1] + tuple2[1]
```

```
# Calculating Sum(ai - 1) * bi
```

```
sum_ai_minus_1_bi = (tuple1[0] - 1) * tuple1[1] + (tuple2[0] - 1) *  
tuple2[1]
```

```
# Calculating Sum ai
```

```
sum_ai_new = tuple1[0] + tuple2[0]
```

```
# Calculating the new formula
result_new = sum_bi_new - (sum_ai_minus_1_bi / sum_ai_new)
result_new
```

```
Result = 2.142857142857143
Rounded to: 2
```


Aggregate function for calculating the identified influence:
High IDV : Retrospective / Learning Loop $\rightarrow \{(0.75, 1)\}$

Defining the tuples

```
tuple1 = (1, 1) #Influence from MoCA v 1.3
```

```
tuple2 = (0.75, 1) #Influence from Ger-CompB Case
```

```
# Calculating the sum of ai*bi
```

```
sum_ai_bi = tuple1[0] * tuple1[1] + tuple2[0] * tuple2[1]
```

```
# Calculating the sum of bi
```

```
sum_bi = tuple1[1] + tuple2[1]
```

```
# Calculating the required formula: Sum(ai*bi) / Sum(bi)
```

```
result = sum_ai_bi / sum_bi
```

```
# Result of the formula: 0.875
```

```
# Calculating Sum bi
```

```
sum_bi_new = tuple1[1] + tuple2[1]
```

```
# Calculating Sum(ai - 1) * bi
```

```
sum_ai_minus_1_bi = (tuple1[0] - 1) * tuple1[1] + (tuple2[0] - 1) *  
tuple2[1]
```

```
# Calculating Sum ai
```

```
sum_ai_new = tuple1[0] + tuple2[0]
```

```
# Calculating the new formula
```

```
result_new = sum_bi_new - (sum_ai_minus_1_bi / sum_ai_new)
```

```
result_new
```

```
Result = 2.142857142857143
```

```
Rounded to: 2
```